

Week 3: Advanced SQL — Window Functions, CTEs, and Advanced Queries

Phase 2: PostgreSQL Mastery | Week 3 of 12

Week Overview

This week covers the most powerful SQL constructs that differentiate senior engineers from juniors: window functions, Common Table Expressions (CTEs), and advanced query patterns. These techniques appear in virtually every data engineering and backend engineering interview. By Friday you will solve problems that previously required multiple queries in a single elegant statement.

Focus: Window functions are the most asked-about topic in PostgreSQL interviews. Master them completely.

Learning Objectives

By the end of this week, you will be able to:

- Write window functions with `OVER()`, `PARTITION BY`, and `ORDER BY`.
 - Use ranking functions: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`.
 - Use value functions: `FIRST_VALUE`, `LAST_VALUE`, `LAG`, `LEAD`, `NTH_VALUE`.
 - Use aggregate window functions: `SUM`, `AVG`, `COUNT` as windows.
 - Define frame clauses: `ROWS BETWEEN`, `RANGE BETWEEN`.
 - Write recursive and non-recursive CTEs.
 - Use CTEs for query decomposition and readability.
 - Write advanced query patterns: pivots, unpivot, top-N per group.
-

Required Reading

- `04_Advanced_SQL/` — All files
-

Daily Schedule

Monday — Window Function Fundamentals (60 min)

Topics:

- Window function syntax: `func() OVER (PARTITION BY ... ORDER BY ...)`
- How window functions differ from `GROUP BY` (rows are NOT collapsed)
- `ROW_NUMBER`, `RANK`, `DENSE_RANK` differences
- `NTILE` for percentile bucketing

```
-- Setup: use the employees table from Week 1-2

-- Row number within each department
SELECT
    first_name,
    department,
```

```

    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employees;

-- RANK vs DENSE_RANK (with a tie in salary)
INSERT INTO employees (first_name, last_name, email, salary, department)
VALUES ('Henry', 'Park', 'henry@co.com', 75000, 'Engineering');

SELECT
    first_name, salary, department,
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank,
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dense_rank,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employees WHERE department = 'Engineering';

-- NTILE: divide into quartiles
SELECT
    first_name, salary,
    NTILE(4) OVER (ORDER BY salary) AS quartile
FROM employees;

```

Tuesday — Value Window Functions (90 min)

Topics:

- LAG and LEAD for comparing adjacent rows
- FIRST_VALUE and LAST_VALUE
- Frame clause: ROWS BETWEEN N PRECEDING AND CURRENT ROW
- Running totals and moving averages

```

-- LAG: compare each employee's salary to the previous employee
SELECT
    first_name, salary,
    LAG(salary, 1) OVER (ORDER BY salary) AS prev_salary,
    salary - LAG(salary, 1) OVER (ORDER BY salary) AS salary_diff
FROM employees ORDER BY salary;

-- LEAD: see the next value
SELECT
    first_name, salary,
    LEAD(salary, 1) OVER (ORDER BY salary) AS next_salary
FROM employees ORDER BY salary;

-- Running total of salary (ordered by hire_date)
SELECT
    first_name, hire_date, salary,
    SUM(salary) OVER (ORDER BY hire_date ROWS UNBOUNDED PRECEDING) AS
running_payroll
FROM employees ORDER BY hire_date;

-- Moving average (3-row)

```

```

SELECT
    first_name, salary,
    ROUND(AVG(salary) OVER (
        ORDER BY id ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ), 2) AS moving_avg_3
FROM employees;

-- FIRST_VALUE and LAST_VALUE per department
SELECT
    first_name, department, salary,
    FIRST_VALUE(salary) OVER (
        PARTITION BY department ORDER BY salary DESC
    ) AS highest_in_dept,
    LAST_VALUE(salary) OVER (
        PARTITION BY department
        ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS lowest_in_dept
FROM employees;

```

Wednesday — Common Table Expressions (CTEs) (90 min)

Topics:

- Non-recursive CTE: query decomposition
- Multiple CTEs in one query
- Recursive CTEs for hierarchy traversal
- CTE vs. subquery performance considerations

```

-- Simple CTE: department salary stats
WITH dept_stats AS (
    SELECT
        department,
        AVG(salary) AS avg_salary,
        MAX(salary) AS max_salary,
        COUNT(*) AS headcount
    FROM employees
    GROUP BY department
)
SELECT * FROM dept_stats WHERE avg_salary > 65000;

-- Multiple CTEs: chained analysis
WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 70000
),
dept_summary AS (
    SELECT department, COUNT(*) AS count, SUM(salary) AS total
    FROM high_earners
    GROUP BY department
)
SELECT d.name AS dept, ds.count, ds.total

```

```

FROM departments d
JOIN dept_summary ds ON d.name = ds.department;

-- Recursive CTE: management hierarchy
ALTER TABLE employees ADD COLUMN manager_id INTEGER REFERENCES employees(id);
UPDATE employees SET manager_id = 3 WHERE id IN (1, 2, 4, 5);
UPDATE employees SET manager_id = 1 WHERE id = 7;

WITH RECURSIVE org_chart AS (
    -- Base: top-level (no manager)
    SELECT id, first_name, last_name, manager_id, 0 AS depth,
           first_name::TEXT AS path
    FROM employees WHERE manager_id IS NULL

    UNION ALL

    SELECT e.id, e.first_name, e.last_name, e.manager_id,
           oc.depth + 1,
           oc.path || ' > ' || e.first_name
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT REPEAT(' ', depth) || first_name AS hierarchy, path, depth
FROM org_chart ORDER BY path;

```

Thursday — Advanced Query Patterns (60 min)

Topics:

- Top-N per group (window + CTE pattern)
- Pivot-style conditional aggregation
- FILTER clause in aggregations
- CASE WHEN in SELECT and WHERE

```

-- Top-2 earners per department
WITH ranked AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn
    FROM employees
)
SELECT first_name, department, salary
FROM ranked
WHERE rn <= 2
ORDER BY department, salary DESC;

-- Pivot: salary distribution by department (conditional aggregation)
SELECT
    COUNT(*) FILTER (WHERE department = 'Engineering') AS eng_count,
    COUNT(*) FILTER (WHERE department = 'Marketing') AS mkt_count,
    COUNT(*) FILTER (WHERE department = 'HR') AS hr_count,
    AVG(salary) FILTER (WHERE department = 'Engineering') AS eng_avg_salary,

```

```

    AVG(salary) FILTER (WHERE department = 'Marketing') AS mkt_avg_salary
FROM employees;

-- Gap detection: find missing IDs
WITH id_series AS (
    SELECT generate_series(1, MAX(id)) AS expected_id FROM employees
)
SELECT expected_id AS missing_id
FROM id_series
LEFT JOIN employees ON expected_id = employees.id
WHERE employees.id IS NULL;

```

Friday — Review + Window Function Mastery Project (45 min)

Mini-Project: Sales ranking and trend analysis.

```

-- Create a monthly sales table
CREATE TABLE monthly_sales (
    month DATE PRIMARY KEY,
    revenue NUMERIC(12,2),
    orders INTEGER
);

INSERT INTO monthly_sales VALUES
    ('2024-01-01', 12000, 120), ('2024-02-01', 15000, 148),
    ('2024-03-01', 11000, 105), ('2024-04-01', 18000, 175),
    ('2024-05-01', 22000, 210), ('2024-06-01', 19500, 188);

-- Write these queries:
-- 1. Month-over-month revenue change (LAG)
-- 2. 3-month moving average revenue
-- 3. Running total revenue
-- 4. Rank each month by revenue
-- 5. Percentage of total revenue per month (window SUM)
-- 6. Highest and lowest revenue months

```

Practice Tasks

1. Write a query that returns only the highest-paid employee per department (no ties).
2. Use a recursive CTE to find all direct and indirect reports of a specific manager.
3. Calculate the 3-month moving average of sales using LAG and window functions.
4. Write a query that returns the "salary percentile" for each employee (0-100).
5. Use FILTER clause to count employees by salary band in a single query.
6. Find employees whose salary is above their department's median (advanced: use PERCENTILE_CONT).
7. Detect the month with the largest month-over-month drop in sales.
8. Write a query that pivots department names into columns showing headcount.

Self-Assessment Checklist

- ☐ I can explain the difference between RANK and DENSE_RANK
 - ☐ I can write LAG/LEAD queries to compare rows
 - ☐ I understand ROWS BETWEEN frame clauses
 - ☐ I can decompose complex queries using multiple CTEs
 - ☐ I can traverse a hierarchy with a recursive CTE
 - ☐ I can implement top-N per group without using LIMIT
 - ☐ I completed the sales trend analysis mini-project
 - ☐ I can write a window function without referencing documentation
-

Mock Interview Questions

1. What is the difference between a window function and GROUP BY?
 2. Explain the difference between RANK() and DENSE_RANK() with an example.
 3. Write a query to find the second-highest salary in each department.
 4. What is a CTE and when would you use one instead of a subquery?
 5. How do you calculate a 7-day moving average in SQL?
 6. Write SQL to find month-over-month growth percentage.
 7. What does PARTITION BY do in a window function?
 8. Explain a use case for a recursive CTE.
 9. How would you use LAG to detect when a user's session started?
 10. Write a query that assigns percentiles to all customers by their total spend.
-

Resources

- This repo: 04_Advanced_SQL/
- Window function visual: <https://www.postgresql.org/docs/16/tutorial-window.html>
- Practice: Mode Analytics SQL Tutorial (window functions section)